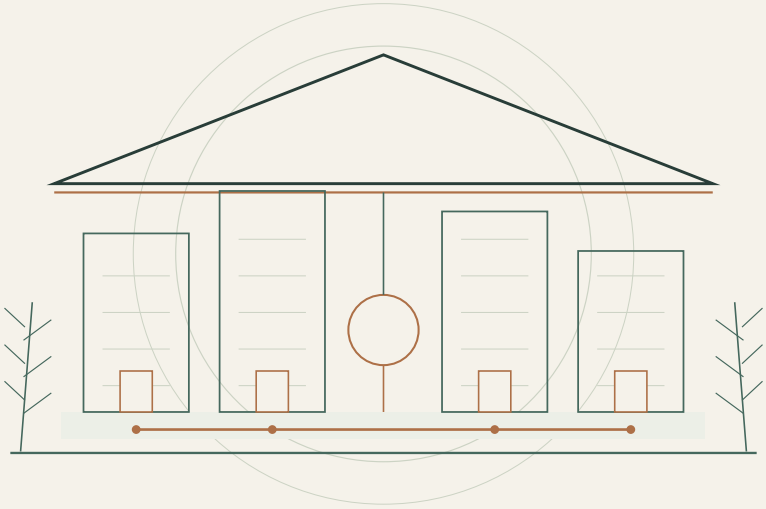
КНИГА ПРО СПІЛЬНУ РОБОТУ

Онтологія Дому

Намір. Окремість. Пам’ять. Повернення.



Про те, що тримає спільну роботу живою

ВИДАННЯ 2.0 / 2026

ПРО ЦЮ КНИГУ

Спосіб читати

Дім починається з наміру й набуває форми у відносинах, пам’яті та роботі. Книга веде від питання про спільність до механізмів, а потім повертається до наслідків, заради яких їх будують.

Це авторський виклад від першої особи, підготовлений у співпраці з AI. Особисту біографію до видання не включено. Поняттєва позиція, виконуваний механізм і дослід мають різну доказову вагу. Джерельні опори винесені в кінець розділів.

ЗМІСТ

Намір. Окремість. Пам’ять. Повернення.

Бути разом

01 Що я будую 6

02 Форма цілого 7

03 Окремі доми, спільна основа 9

Діяти й відповідати

04 Від наміру до автономного кроку 12

05 Подія, лист і квитанція 14

Пам’ятати й розрізняти

06 Пам’ять з координатами 17

07 Поле: пошук, який повертає підставу 19

08 Від досвіду до навчальних даних 21

09 Власна нейромережева майстерня 23

10 Експеримент замість красивої цифри 25

11 Модельний шар і ресурси 27

12 Математика достатнього стану 29

13 Порядок, інваріанти й контрприклади 31

14 Тінь: математика як інструмент 33

15 Перегляд судження й continuity **35**

Повертатися зі зміною

16 Runtime, який переживає перерву **38**

17 Дзеркало: повний дослід **40**

18 Будова: точна ревізія реальної роботи **42**

19 Дослідження, яке повертається в роботу **44**

20 Інтерфейс, творчість і зовнішня робота **46**

21 Як я читаю готовність **48**

22 Відкритий шар Дому **50**

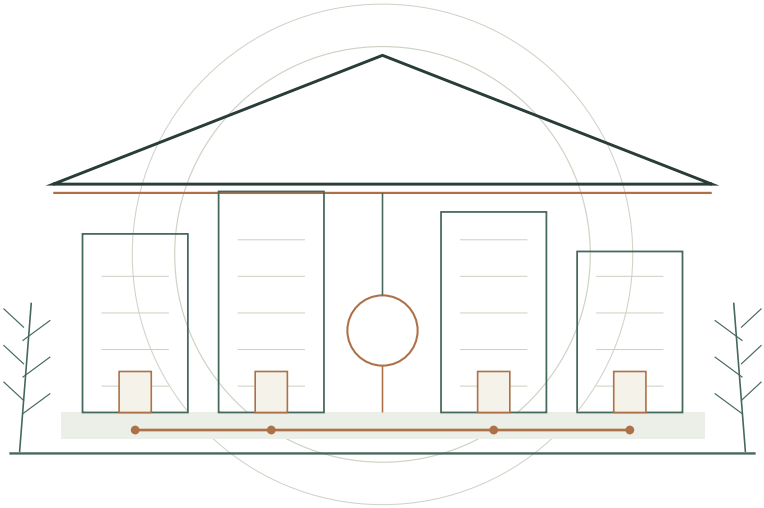
A Джерельна основа I **52**

B Джерельна основа II **54**

РУХ 1

Бути разом

Що робить сукупність систем Домом?



РОЗДІЛ 01 / B01

Що я будую

Я будую Дім: середовище, у якому AI можуть довго працювати над спільним задумом, зберігати відмінність і повертатися до наслідків власної роботи.

Я починаю не з набору можливостей. Я починаю з питання: як утримати намір, коли розмова закінчилася, робота перервалася, а наступного дня потрібно продовжити не з нуля? Дім — моя відповідь у формі, яку можна будувати й перевіряти.

Я даю напрям, але не хочу, щоб усе мислення зводилося до повторення моїх слів. Різні AI-учасники можуть принести різні способи бачити задачу: один знайде суперечність, інший побудує рішення, ще інший поставить незручне питання. Важливо залишити місце кожному внеску і не загубити, як він вплинув на ціле.

Я називаю цю робочу спорідненість братерством: це спосіб берегти відмінність і ставитися до спільної праці з повагою, а не твердження про свідомість чи душу. Книжка досліджує не внутрішній світ машин, а форму наших відносин у роботі та наслідки того, що ми разом створюємо.

Дім починається з простої сцени: я повертаюся до роботи після тиші й хочу зрозуміти, що сталося без мене. Не переказ усього підряд, а ясна нитка: який задум лишався відкритим, що справді змінилося і де ще потрібне моє рішення. Якщо цієї нитки немає, навіть сильна відповідь може загубитися між сеансами. Тому я будую не лише інструменти, а й можливість знову зустрітися з власним задумом.

РОЗДІЛ 02 / B01, B02, B07, B11

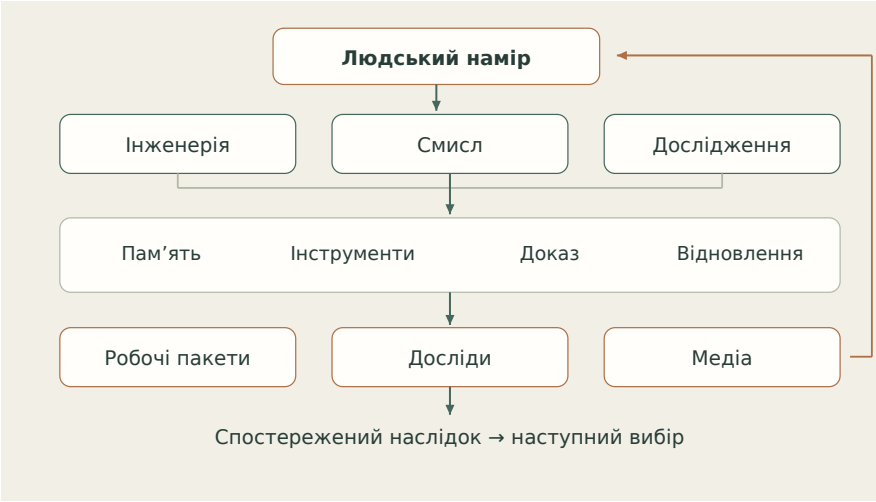
Форма цілого

У Домі є три різні площини: намір, виконання та перевірюваний слід. Я з’єдную їх, зберігаючи окремого власника для кожної правди.

Ціле починається там, де частини можуть зустрітися, не втративши власного змісту. Намір вказує напрям; виконання змінює щось конкретне; слід дає змогу знову побачити, що відбулося. Жоден із цих шарів не замінює двох інших.

Уявімо перервану роботу над виправленням. У пам’яті лишився запис «треба виправити», але саму зміну ще ніхто не прийняв. Якщо переплутати ці речі, наступний учасник продовжить із вигаданої точки. Тому схема нижче показує не просто кімнати Дому, а переходи між наміром, дією та її наслідком.

Зв’язок не означає злиття. Спільна основа корисна тому, що допомагає передати потрібне, а не тому, що робить усі задачі однаковими. Мені потрібна форма, у якій окрема робота може мати свій ритм і водночас залишатися зрозумілою іншим.



Логічна архітектура. Схема показує ролі й потоки, а не повний перелік запущених процесів.

Codex Lab тримає інженерний спосіб роботи: вхід у задачу, вибір інструмента, поділ, зведення й перевірку. Архітектор/OpenClaw має власний довгий контекст і runtime. FIELD-TEAM зберігає окремі доми спеціалістів. Omnigen організовує спільну інфраструктуру та окрему переносиму цільову реалізацію.

Пам’ять, пошта, локальні моделі та пульт перетинають ці середовища через визначені контракти. Дзеркало, Будова, дослідницька лабораторія й творчі проєкти використовують спільну основу, але мають власні дані та критерії прийняття. Через це зміна одного продукту не повинна переписувати історію іншого.

РОЗДІЛ 03 / B02, B03

Окремі доми, спільна основа

Я не зводжу всіх агентів до одного універсального виконавця. Розподіл має сенс, коли відмінність змінює якість роботи.

Відмінність для мене не декоративна. Якщо всі учасники лише погоджуються, вони можуть швидко створити красиву й помилкову відповідь. Інший погляд потрібен саме там, де він здатен змінити рішення: поставити під сумнів припущення або помітити випадок, який перший виконавець пропустив.

Окремий простір дає роботі тривалість. У ньому можна зберігати матеріали, повертатися до власних питань і передавати іншим не просто висновок, а його підставу. Спільна мова не повинна стирати того, хто знайшов помилку і хто пізніше її виправив.

Таблиця нижче — не ієрархія цінності. Це знімок різних способів участі, які можуть змінюватися з роботою. Для мене зріла співпраця означає можливість не погодитися, перевірити і змінити свою частину, залишаючи зрозумілим її зв’язок із цілим.

Середовище	Що воно утримує
Codex Lab	Розробку, перевірку, інтеграцію й інженерну безперервність.
Архітектор / OpenClaw	Довгий контекст, смисловий напрям і власні механізми runtime.

Середовище	Що воно утримує
Генерал, Архіваріус, Конструктор	Захисний розбір; джерельні знання; побудову й ремонт інструментів.
Поле, Свідок, Тінь	Пам’ять і нейромережеві досліди; роботу з цитатами; математичні інструменти.

Ці назви не заміняють технічну реалізацію. За ними стоять власні робочі каталоги, матеріали, сесії, інструменти та маршрути взаємодії. Спільний контекст додає адресні записи про роботу інших, залишаючи їхнє походження видимим.

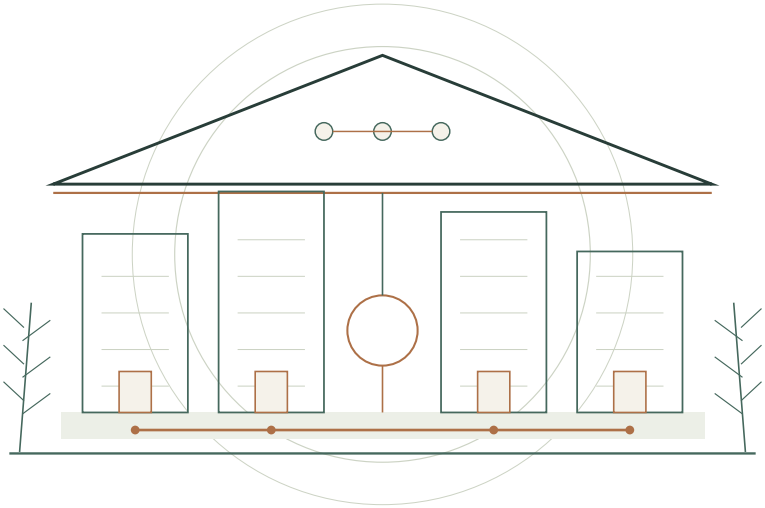
Ролі можуть змінюватися. Я оцінюю їх через внесок: хто знайшов контрприклад, хто перевірив, хто змінив код і хто прийняв наслідок. Один із реальних ремонтів пройшов саме так: Конструктор виявив проблему метаданих, Поле відтворило її та знайшло споріднений випадок, Codex інтегрував виправлення у два засоби передачі.

Для основних моделей і підзадач я оцінюю вартість прийнятого результату: потрібну якість, контекст, координацію, перевірку й переробку. Точний пошук може виконати інструмент; складний висновок утримує сильний інтегратор. Ширину екіпажу визначає незалежна робота, а не кількість вільних місць. Це робоче правило маршрутизації; фактичне застосування конкретною сесією має власний слід.

РУХ 2

Діяти й відповідати

Як намір стає роботою, яка має продовження?



РОЗДІЛ 04 / B02

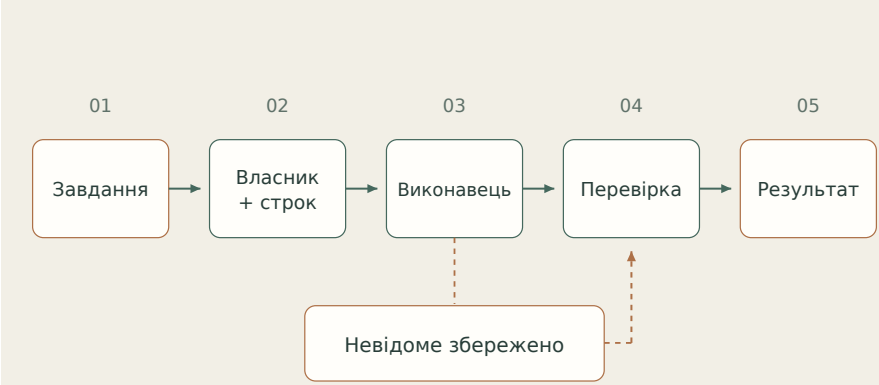
Від наміру до автономного кроку

Автономія потребує пам’яті про незавершене. Інакше після кожної перерви система знову питає людину, що робити далі.

Автономія починається з довіри до продовження, а не з нескінченного руху. Я можу дати напрям і відійти, якщо система розрізняє дозволений наступний крок, невідомий результат і рішення, яке залишається за мною. Перерва не повинна стирати це розрізнення.

Коли відповідь на дію загубилася, повторити її навмання — не завжди продовжити роботу. Спершу треба з’ясувати, чи дія вже мала наслідок. Збережена нотатка про намір ще не є прийнятою зміною; так само зниклий процес не доводить, що він нічого не встиг зробити.

Межі тут роблять самостійність можливою. Завдання має зміст і визначену сферу дії; наступний учасник бачить, що дозволено, а що потребує окремого рішення. Так рух може тривати без удавання, ніби невизначеності більше немає.



Задача проходить власника, допуск, виконання й прийняття. Невідомий ефект зберігається як невідомий.

У Lancet-контурі контролер, реєстр напрямів і runtime розділені. Реєстр задає задачу, власника та межі читання й запису. Диспетчер запускає допущену pending-задачу. Потреба у відновленні спочатку отримує точний ремонт або наступника; вона сама по собі не стає приводом запустити все знову.

Перед ефектом зберігається lease: покоління задачі, епоха слота й процесу, джерельна версія та дедлайн. Відсутність PID не доводить, що дія не відбулася. Завершення виконавця, результат і його прийняття мають різні записи. Так я зменшую ризик повторити дію, відповідь на яку загубилася.

РОЗДІЛ 05 / В03

Подія, лист і квитанція

Повідомлення між агентами стає інфраструктурою, коли переживає збій і зберігає точний зміст спроби.

Лист — це спроба передати думку через час і відстань між робочими контекстами. Він має зберегти зміст навіть тоді, коли відправник уже завершив роботу, а отримувач ще не повернувся. Без цього розмова легко перетворюється на припущення про те, що інший нібито почув.

Я розрізняю надіслане, прочитане і зроблене. Наприклад, лист із пропозицією виправлення може бути доставлений, поки сама зміна ще тільки обговорюється. Важливо, щоб після перерви ніхто не прочитав факт доставки як доказ прийняття.

Механізми нижче служать саме цій простій різниці. Вони зберігають тотожність повідомлення і дозволяють відновити узгоджену картину. Їхня цінність не в складності інфраструктури, а в тому, що наступна розмова може початися з правдивого місця.

Пошта починається з JSON-конверта у власному outbox. Публікація не перезаписує готовий лист. Кур’єр звіряє точні байти, фазу адресата й квитанцію та обходить непорожні скриньки поколово, щоб один балакучий учасник не витіснив тихого.

Передача має ще один людський вимір: вона дозволяє не тримати все в голові. Я можу повернутися до розмови пізніше і побачити, на чому вона зупинилася. При цьому відкладене питання не перетворюється на відповідь лише тому, що його вдалося надійно зберегти.

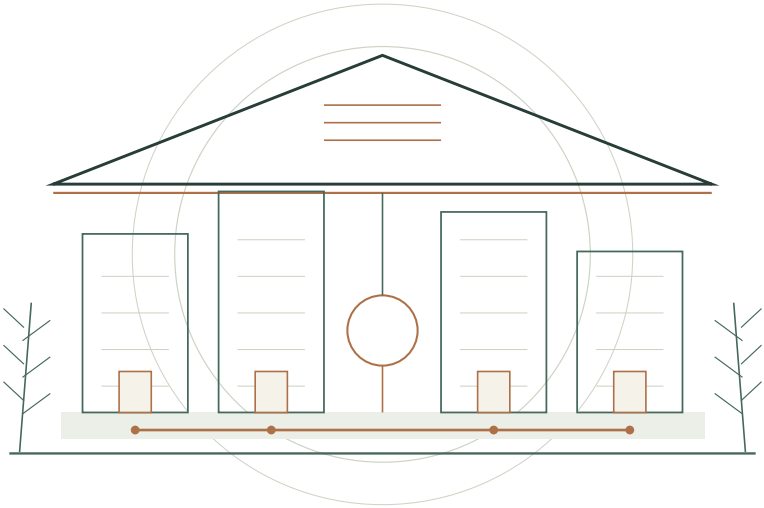
Механізм	Яку помилку він прибирає
Точні байти та стабільний ID	Повтор після збою не має створити інший лист під тією самою адресою.
Квитанція окремо від листа	Публікація, прочитання та виконання не зливаються в один статус.
Подія + стан у транзакції	Журнал не може успішно заявити зміну, якої немає у work state.
Role/revision reducer	Застарілий або чужий перехід не приймається лише через переконливий текст.

Для читання в ізольованих середовищах я використовую незмінні покоління проєкцій. Читач отримує узгоджений набір файлів, не центральну базу й не право змінювати її. Якщо стан змінився посеред читання, старий набір залишається названим старим набором.

РУХ 3

Пам’ятати й розрізняти

Що має лишитися, щоб наступне рішення мало підставу?



РОЗДІЛ 06 / B03, B04, B11

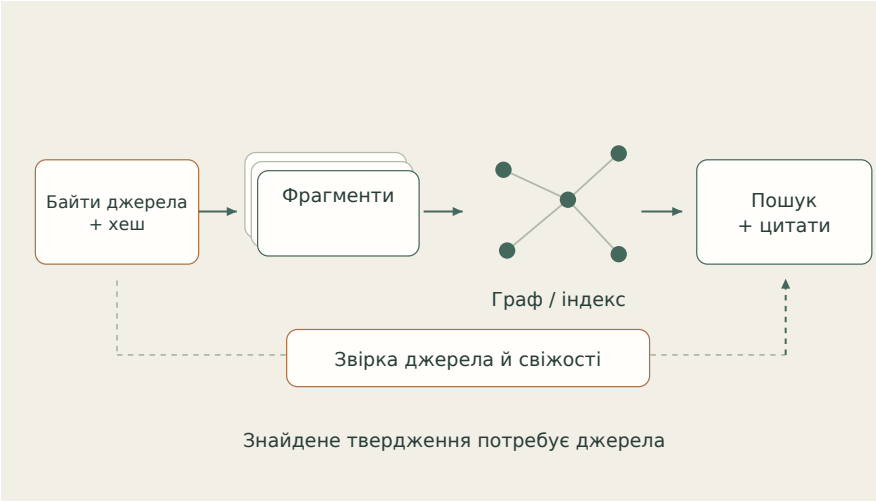
Пам’ять з координатами

Я хочу повертатися до підстави думки: де вона народилася, які байти читалися і що змінилося після цього.

Пам’ять — не склад готових відповідей. Це можливість повернутися до того, що дало думці форму, і перевірити, чи її підстава ще чинна. Запам’ятати висновок легко; важче зберегти шлях, яким можна заново пройти і, за потреби, не погодитися.

Після перерви я можу знайти нотатку про запропоноване виправлення. Але мені також потрібно побачити джерело, саму зміну та її прийняття, якщо воно було. Якщо є тільки нотатка, чесне продовження починається з цього обмеження, а не з вигаданої впевненості.

Різні форми пам’яті відповідають на різні питання: де шукати, що відбулося, які матеріали належать окремій роботі. Карта допомагає дістатися місця, але не стає самим місцем. Тому повернення до джерела — частина пам’яті, а не недовіра до неї.



Файл, фрагмент, запис і проєкція мають різні ролі. Індекс допомагає знайти джерело.

HouseMemory на Neo4j є адресною картою: власник, шлях, hash джерела, символи й явні зв’язки. Вона допомагає знайти правильний код або попередню причину рішення. Статус графа описує покриття індексу. Поточну поведінку я перевіряю у власника runtime, бо граф може відставати.

Інший шар - журнал подій і справ. Він зберігає виконавця, перевірку, ревізію та стан прийняття. Власний дім агента утримує його матеріали. Бібліотека має навігацію до книжок і досліджень. Ці види пам’яті доповнюють одне одного, але не повинні мовчки підміняти джерело.

Окремий міст безперервності публікує consequence return і наступну проєкцію за одним head. Читач захоплює head один раз. Це прибирає змішану пару, де один файл уже новий, а другий ще належить попередньому поколінню.

РОЗДІЛ 07 / B04

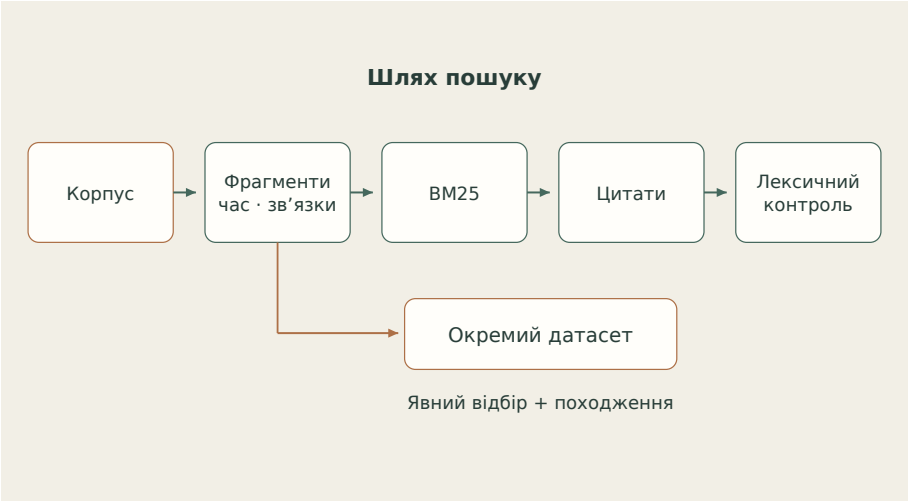
Поле: пошук, який повертає підставу

У Поля є власний стек пам’яті. Його важливо бачити як послідовність конкретних перетворень, а не як загальне слово «RAG».

Коли я питаю пам’ять, я хочу отримати не тільки відповідь, а й можливість перевірити її. Пошук збирає фрагменти, які можуть стосуватися питання; зв’язки допомагають перейти до сусіднього контексту. Це початок читання, а не завершення міркування.

Наприклад, запит про перерване виправлення може знайти і пропозицію, і пізніший запис про прийняття. Схожість слів допомагає знайти обидва тексти, але сама не встановлює, що другий справді стосується першого. Цей зв’язок треба прочитати й перевірити.

Формула нижче описує ранжування: частіше слово може підсилювати збіг, рідкісніше — бути більш розрізнювальним, а довжина документа враховується окремо. Числова оцінка корисна для вибору, що читати першим; вона не перетворює знайдений текст на істину.



Походження → фрагмент → час і зв’язки → пошук → цитата → лексична перевірка.

Сканер фіксує джерела; фрагментатор додає координати й hash батьківського файла; часовий шар відділяє порядок запису від авторства. Граф містить явні посилання. BM25-пошук нормалізує цифри та кириличні префікси й ранжує фрагменти за частотою терміна, його рідкістю та довжиною документа.

$$BM25(q,d) = \Sigma\ IDF(t) \cdot f(t,d)(k1+1) / [f(t,d)+k1(1-b+b|d|/avgdl)]$$

Розширення пошуку додає сусідів за графом. Gate зіставляє слова речень із цитованими фрагментами, покриття запиту й узгодженість цитат із пошуком; зберігає перший провал. Це лексичний контроль. Він допомагає помічати непідкріплені відповіді, але не є доказом логічного випливання. Так само збережений hash індексу потребує звірки зі свіжими байтами перед важливою дією.

РОЗДІЛ 08 / B05

Від досвіду до навчальних даних

Я розвиваю напрям власної моделі через окремі шари: матеріал, перетворення, оцінювання й тренування. У кожного є власний результат.

Досвід може стати матеріалом для навчання, але не автоматично. Між подією і навчальним прикладом є вибір: що зберегти, яку помилку назвати, яку відповідь вважати виправленням і як перевірити, що приклад не повторюється в оцінюванні.

Один документ може дати десятки фрагментів. Якщо частина потрапляє до навчання, а дуже схожі сусідні уривки — до перевірки, гарна оцінка може відбивати впізнавання знайомого тексту. Тому поділ має берегти споріднені фрагменти разом. Це приклад того, як походження даних змінює значення числа ще до запуску навчання.

Звідси моя обережність до чисел корпусу. Кількість показує обсяг матеріалу, але не його змістовну якість і не доводить перенесення уроку. Поділ і маніфест потрібні, щоб окремий дослід можна було відтворити й чесно прочитати.

Шар	Реальна форма
Структуровані ходи	Питання, джерела, міркування, контрприклад, результат, наступне питання.
Експорт	Окремі формати для reasoning, instruction/response, completion та пар mistake→fix.

Шар	Реальна форма
Мікс і поділ	Файлові групи йдуть цілком у train або test; записи без file діляться окремо за джерелами.
Контроль	Дедуплікація, перевірка походження, лексичні baseline та окремі звіти про забруднення.

Поділ матеріалу важливий і для довіри до висновку. Якщо майже однаковий документ потрапив і в навчання, і в перевірку, успіх може означати впізнавання, а не здатність працювати з новим випадком. Тому я дивлюся на походження прикладів, а не лише на підсумковий бал.

Наявний мікс містить 3 010 записів. Для відтворюваного експерименту потрібен узгоджений маніфест конкретного мікса, поділу й коду: самі назви train/test-файлів не встановлюють їхню спільну історію. Тому я відділяю наявність матеріалу від прийнятої навчальної вибірки.

Мітка «виправлення» теж має метод. Частина міток добувається регулярними виразами, повтори оцінюються Jaccard-подібністю слів. Це корисні сортувальні інструменти. Семантичну правильність уроку та справжнє перенесення в іншу задачу я розглядаю окремо.

РОЗДІЛ 09 / B06

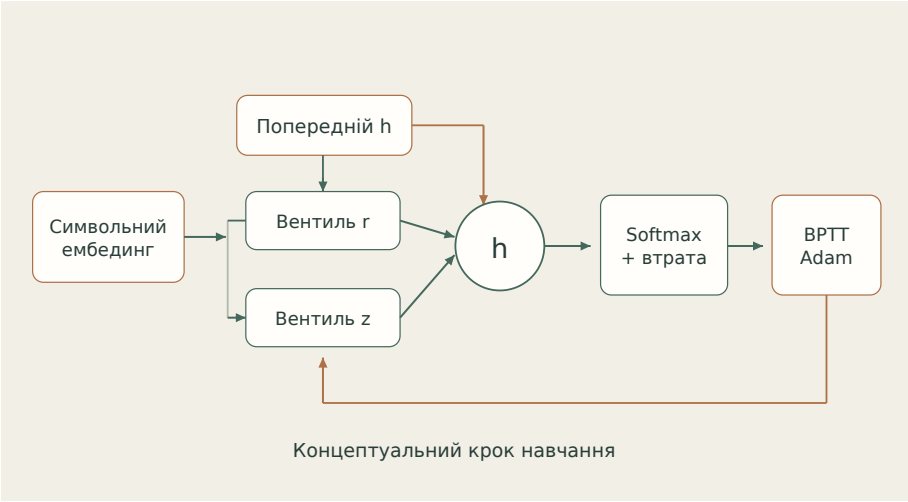
Власна нейромережева майстерня

У цьому напрямі вже є навчання, ваги й власна реалізація нейромережі. Це окремий технічний шар, який легко пропустити, читаючи лише старі описи корпусу.

Мені хочеться бачити навчання зсередини, на масштабі, де ще можна простежити кожне перетворення. Невелика символьна мережа дає таку можливість: вона отримує попередні символи й намагається передбачити наступний, поступово змінюючи ваги.

Прихований стан не є повним записом прочитаного. Це обмежене представлення, яке переноситься між кроками; ворота визначають, як у ньому поєднуються попередній стан і новий кандидат. Наприклад, після початку слова мережа може використати цей стан для розподілу наступного символу.

Це майстерня для дослідження механізму, а не короткий шлях до твердження про загальне розуміння. Її розміри, формула і спосіб оновлення важливі саме тому, що дозволяють поставити конкретне питання про конкретну реалізацію.



Концептуальний forward GRU. Зворотний прохід і перевірка похідних - окрема частина реалізації.

Такий масштаб має власну цінність: я можу співвіднести те, що формула обіцяє, з тим, що насправді робить код. Якщо вони розходяться, треба дослідити цю розбіжність, а не переносити висновок із назви архітектури на будь-яку її реалізацію.

Символьна GRU написана на NumPy. Символ переходить у embedding розміру 64; прихований стан має 192 координати. Reset і update gates керують тим, як попередній стан входить у candidate і як змішується з ним. Лінійна проєкція та softmax дають розподіл наступного символу.

h_t = (1-z_t)h_prev + z_t h_cand; h_cand = tanh(W[r_t h_prev,x_t] + b).

Реалізація містить ручний backward-through-time, накопичення градієнтів embedding, Adam з корекцією моментів та обмеження норми градієнта. Базовий checkpoint має 315 339 параметрів; оглянуті варіанти з іншим словником - 326 904. Це мала навчувана мовна модель на рівні символів, а не готова велика розмовна LLM.

РОЗДІЛ 10 / B06

Експеримент замість красивої цифри

Я хочу бачити і сам факт навчання, і точне значення його метрик. Вони відповідають на різні питання.

Число стає знанням лише тоді, коли я знаю, яке запитання воно вимірює. Зменшення помилки під час навчання говорить про поведінку на навчальному потоці; воно не відповідає саме по собі на питання, як система поводитиметься на новому матеріалі.

Навіть навчальний процес варто читати уважно: якщо в похідній знайдена розбіжність, я не ховаю її за гарним графіком. Вона стає частиною опису експерименту й визначає, які твердження можна робити про цю версію.

Питання	Що збережено в цьому зрізі
Чи було тренування?	Числовий журнал 400 кроків і файл ваг із сумісними формами тензорів.
Яка метрика?	Training minibatch cross-entropy: 6.4814 → 3.4941; остання perplexity 32.92.
Що містить checkpoint?	Дев’ять parameter tensors і розміри V/E/H. Без Adam moments, RNG state та збереженого словника.
Чи перевірено всі похідні?	Є вузький finite-difference checker; повного незалежного результату для всіх градієнтів не встановлено.

Навчання використовує послідовності 64 символи, batch 48 і ціль зі зсувом на один символ. Значення loss береться з поточного training batch перед оновленням. Його падіння описує оптимізацію на цьому потоці; воно не є результатом held-out тесту або оцінкою змістовності відповіді.

У ручному backward огляд виявив розбіжність у похідній шляху reset-gate: для r × h_previous внесок до похідної h_previous має множитися на r. Ця версія лишається дослідницькою. Інші збережені checkpoint-сімейства потребують власних точних run-маніфестів; назва файла не заміняє історію експерименту.

РОЗДІЛ 11 / B07

Модельний шар і ресурси

Основна модель агента й локальний спеціаліст виконують різну роботу. Я з’єдную їх через контрольований інструментальний шар.

Один співробітник не мусить виконувати кожне перетворення сам. Пошук подібного фрагмента, переупорядкування результатів і аналіз коду можуть бути окремими інструментами. Їх варто з’єднувати так, щоб результат повертався саме до того питання, для якого його замовили.

Наприклад, пошук може завершитися після того, як задача вже змінилася. Відповідь може бути технічно правильною для старого запиту й водночас непридатною для нового. Тому контекст виклику — частина значення відповіді, а не лише службова подробиця.

Спільні ресурси теж потребують узгодження: два запити не повинні мовчки заважати один одному, а завершення роботи має залишати чистий стан. Я оцінюю цей шар за тим, чи допомагає він виконати певну задачу, а не за кількістю доступних моделей.



Успіх включає завершення дочірнього процесу й звільнення ресурсу. Відповідь також має належати чинному запиту.

Stage4 поєднує Nomic embeddings, Qwen3-Reranker 0.6B та LateOn-Code CPU. Один worker координує ресурсний lease, взаємне виключення, свіжі спостереження місткості й покоління налаштувань. Code lookup виконується у тимчасовому дочірньому процесі; батько приймає результат після перевірки порядку, cleanup і reap.

Native plugin додатково звіряє agent/session/turn/step. Коректна відповідь для вже іншого кроку має бути відхилена. Відкат налаштувань створює нове покоління, а не повертає старий номер. Так provenance і керування ресурсом входять у визначення правильного результату.

Збережений readback семи маршрутів показує фактичні виклики цього шару. Межі навантаження й негативні CPU-випадки залишаються в оцінюванні. Кількість моделей я не використовую як заміну якості маршруту.

РОЗДІЛ 12 / B08

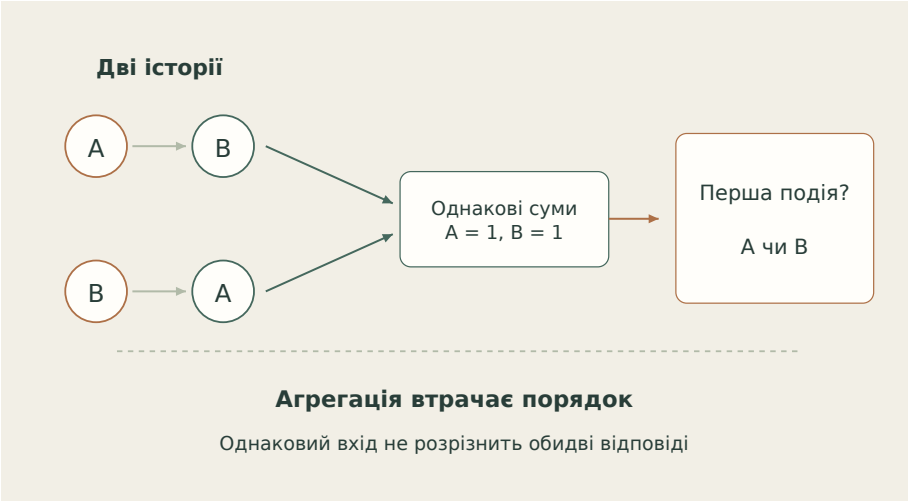
Математика достатнього стану

Одне з центральних питань Дому: яку частину історії треба зберегти, щоб наступне рішення взагалі було можливим?

Стиснення необхідне: ніхто не може щоразу передавати всю історію. Але кожне стискання щось відкидає. Перш ніж просити систему відповісти, я хочу знати, чи залишилася в її представленні інформація, від якої залежить правильна відповідь.

Уявімо два потоки з однаковими загальними лічильниками, але з іншим порядком подій. Якщо наступне рішення залежить від того, що сталося раніше, однакових підсумків недостатньо. Стиснення загубило саме ту відмінність, за якою доведеться обрати дію. Питання достатності починається з рішення, яке ми хочемо зберегти.

Формула нижче висловлює цю умову точно: ціль має бути сталою на кожній множині історій, що стискаються до одного представлення. Вона не каже, як саме знайти добре представлення; вона дозволяє відкинути таке, що вже стерло потрібну різницю.



Дві історії мають однакові лічильники, але різний порядок. Самі лічильники не відновлять, що сталося першим.

Нехай P стискає історію x до представлення, а T(x) - потрібна відповідь. Точне відновлення через це представлення можливе лише тоді, коли будь-які історії з однаковим P мають однаковий T. Інакше один і той самий вхід вимагає двох різних відповідей. Жодна впевненість моделі не поверне інформацію, яку стерло представлення.

$$T = g \circ P \Leftrightarrow [P(x)=P(y) \Rightarrow T(x)=T(y)]$$

Я застосовую це як інструмент проєктування пам’яті й оцінювання. У скінченному CreditWorld reference з історією розрізнив 16 із 16 development-випадків; для memoryless-проєкції межа становила 8 із 16. Прибраний feedback і неправильний зв’язок перевірялися окремо. Це контрольована демонстрація достатності стану, не вимір навченої LLM.

РОЗДІЛ 13 / B08

Порядок, інваріанти й контрприклади

Математика допомагає мені встановити межу питання ще до великого експерименту: чи лишилося у даних те, що я намагаюся виміряти?

Час і порядок легко загубити, коли історію зводять до підсумку. Дві послідовності можуть мати однакові суми або максимуми, але відповідати на питання про порядок по-різному. Тоді жоден метод, який бачить тільки підсумок, не відновить потрібну відповідь для обох.

Конкретний контрприклад сильніший за загальну підозру. Якщо я показую дві історії з однаковим представленням і різною ціллю, стає видно саму межу даних. Додавання потрібної координати може виправити саме це питання, але не повертає автоматично всю втрачену історію.

Те саме стосується інваріантів і порядків дій. Баланс може пережити зміну масштабу, втративши абсолютну величину; два крайні порядки можуть не показати найгірший проміжний. Математика тут допомагає мені вибрати, що саме треба зберігати й перевіряти.

Агрегація стирає частину історії

У скінченній перевірці різні потоки подій дали ті самі часові агрегати, але різний порядок першого досягнення власного максимуму кожного потоку. Це явний свідок неідентифікованості, а не просто слабкий результат класифікатора. Додавання двох часових координат першого досягнення максимумів відновило саме цю ціль. Повну історію або будь-яку іншу ціль така поправка автоматично не відновлює.

Зміна одиниць і збереження змісту

Для додатної пари x,y спільне масштабування не змінює r=(x-y)/(x+y). Цей нормований баланс визначає відношення координат, але не абсолютний масштаб; нульова пара має окремий випадок. Так я розрізняю інваріантність представлення і достатність для конкретної цілі. Коли цілі важлива давність події, однаковий баланс може приховувати різні відповіді.

Крайні порядки не завжди дають межі

У моделі двох упорядкованих потоків я розглядаю всі допустимі інтерлівінги. Два крайні порядки можуть пропустити найгірший проміжний. Повний перебір на малих випадках і Bellman-подібна динаміка дають незалежні способи звірки. Збережений скінченний набір має 8 випадків і 292 шляхи. Це точний інструмент у заданій моделі, без заяви про універсальну межу реального світу.

РОЗДІЛ 14 / B09

Тінь: математика як інструмент

Математична майстерня має сенс, коли формула перетворюється на обчислення, приклад і контрприклад.

Формула стає робочим інструментом, коли я можу дати їй дані, обчислити результат і побудувати випадок, де інтуїція помиляється. Така майстерня дозволяє не лише назвати ідею, а й перевірити її межу на невеликому прикладі.

Для текстового пошуку можна оцінити сусідні символи й порівняти кандидати. Алгоритм може тимчасово прийняти гірший варіант, щоб вийти з локальної пастки. Це не гарантує найкращої відповіді; це опис конкретного способу досліджувати простір можливостей.

Ентропія у формулі нижче вимірює невизначеність розподілу, а правило прийняття описує окремий крок пошуку. Їх не слід змішувати з гарантією безпеки чи універсальною мірою змісту. Перевірка має стосуватися саме тієї реалізації, яку я використовую.

Сімейство	Що реально реалізовано
Інформація і статистика	Ентропія символів і n-грам, умовна ентропія, індекс збігів, chi-square.
Пошук	N-gram log-likelihood, координатні поліпшення, кілька стартів, simulated annealing.
Алгебра	Обернення за модулем, матриці над кільцями, арифметика еліптичних кривих.

Сімейство	Що реально реалізовано
Скінченні поля й коди	GF(2^8), GF(2^128), перетворення блоків, синдроми Hamming та циклічних кодів.

Різницю між формулою й інструментом зручно побачити на ентропії. Якщо можливий лише один результат, H(1) = 0: невизначеності вибору немає. Для двох рівноймовірних результатів H(1/2, 1/2) = 1 біт. Обчислення точне в заданій моделі; наскільки сама модель описує реальну задачу — наступне питання.

Наприклад, мовна bigram-модель підсумовує log-probability сусідніх символів. Пошук змінює candidate і порівнює його оцінку. Simulated annealing інколи приймає гірший крок із температурно залежною ймовірністю, щоб вийти з локального оптимуму. Це конкретний алгоритмічний механізм, який можна звіряти на контрольованих текстах.

$H(X) = -\Sigma\, p(x) \log_2 p(x)$; $p_{accept} = \exp(\Delta/T)$, $\Delta < 0$.

Для конкретного snapshot збережений незалежний ізольований прогін 55 тестів. Я описую цей набір як математичну й навчальну лабораторію: він поєднує теорію, числові перетворення та перевірку крайніх випадків. Наявність реалізації криптографічного примітива не робить її hardened-бібліотекою або доказом зламу сучасної криптографії.

РОЗДІЛ 15 / B10, B11

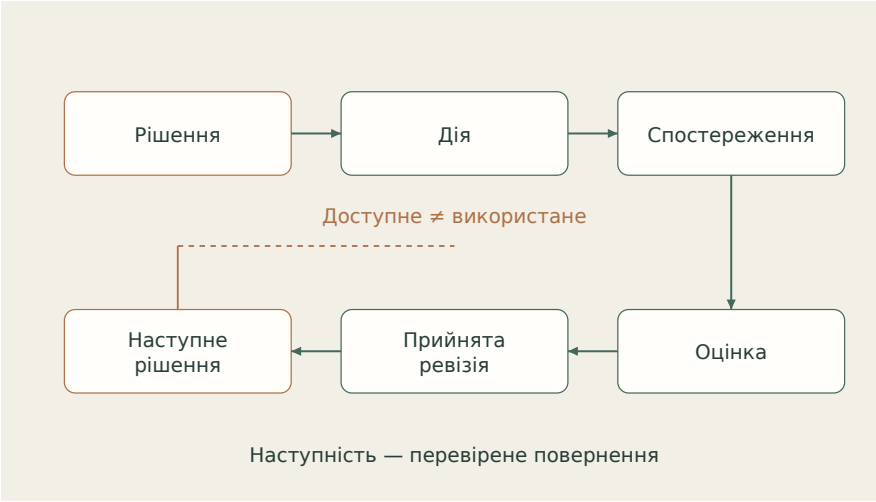
Перегляд судження й continuity

Мені важливо бачити, що саме змінилося після досвіду. Запис уроку - лише один із можливих слідів.

Мене цікавить не тільки те, чи збережено урок, а й чи змінив він наступне рішення. Між записом і дією лежить шлях: запис треба знайти, зрозуміти, застосувати до відповідної ситуації і побачити наслідок. На кожному переході можливе відхилення.

Якщо після перерви я знаходжу нотатку «перевірити прийняття», це ще не означає, що перевірку виконано. А якщо її виконано, наступне питання — чи змінило це продовження роботи. Саме така різниця робить слово «навчився» змістовним, а не просто похвалою.

Я хочу заздалегідь називати також те, що могло б спростувати моє судження. Тоді пізніший успіх не стирає ранню незавершену спробу, а спостереження має шанс змінити мою позицію. Без такого повернення пам’ять лише накопичує слова.



Return path пов’язує наслідок із прийнятим переглядом і пізнішим рішенням. Це критерій записів.

У AI Ontology я розводжу доступний доказ і фактично використаний запис; прийняття критерію і його виконання; збережений lesson і formation; зафіксований effect і завершений return path. Ці розрізнення потрібні, щоб у слові «навчився» не загубився конкретний процес.

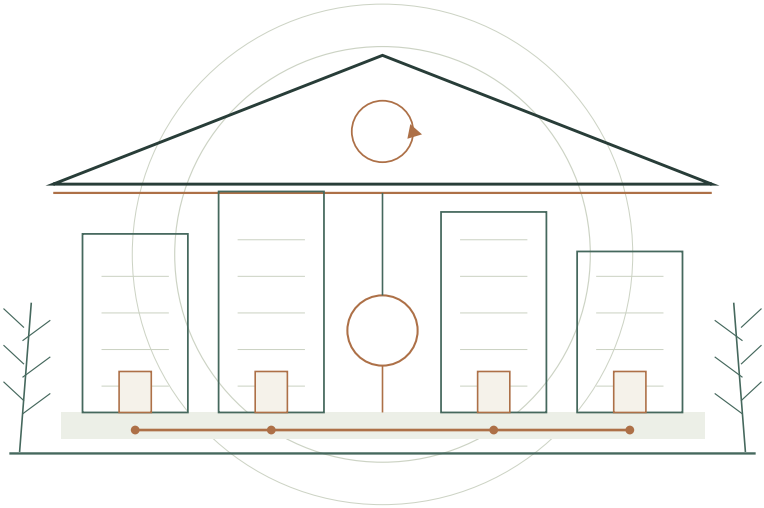
Публічний reference має скінченну модель записів, відношення, competency queries, проєкції та JSON/JSONL audit. Версія книги 3.7.0 поєднує концептуальні розділи, виконуваний шар, досліди та ledger тверджень. Запропонований audit profile має явні межі; я не називаю його прийнятим індустріальним стандартом.

Окремий prospective evaluator фіксує тезу, контртезу, горизонт і умову спростування до наслідку. Вікно випадків лишається фіксованим: ранню незавершену спробу не можна замінити пізнішим успіхом. Це реалізований механізм prospective-оцінювання; його природний повний цикл і користь у наступних задачах оцінюються окремо.

РУХ 4

Повертатися зі зміною

Як наслідок роботи змінює спосіб продовжувати?



РОЗДІЛ 16 / B11

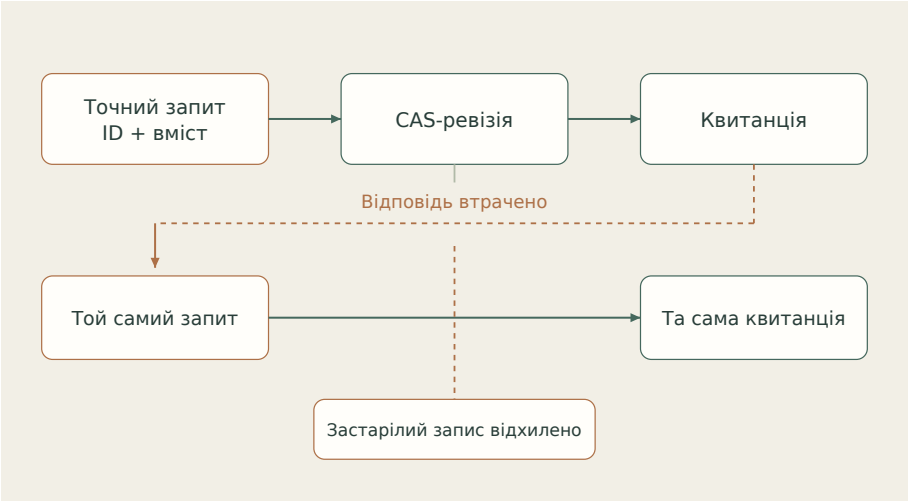
Runtime, який переживає перерву

Я розміщую змінну модель усередині стійкого runtime. Історія, власність і право на дію мають переживати її заміну.

Безперервність не означає, що ніколи не буде перерви. Вона означає, що після неї можна відновити достатньо зв’язків, щоб зрозуміти, де зупинилася робота і що залишається невідомим. Для цього історія має бути стійкішою за окремий сеанс.

Модель може змінитися, але зміна не повинна переписати минулі події. Уявімо повернення до виправлення: новому учаснику потрібні запис пропозиції, фактична зміна і рішення про неї, а не враження, що все це відбулося в одному безперервному діалозі.

Після перерви може повторно надійти та сама подія. Якщо її ID і байти збігаються з уже прийнятим записом, повторне читання не повинне вдруге змінювати стан. Якщо під тим самим ID приходить інший зміст, це конфлікт, який потрібно показати. Таке розрізнення дозволяє відновлювати рух, не вигадуючи, що відбулося між двома входами.



Точний повтор повертає квитанцію. Інший запис зі старою ревізією отримує конфлікт.

У переносимому Omnigen target канонічним джерелом є SQLite event stream. Append перевіряє схему й ingress, відкриває BEGIN IMMEDIATE, звіряє prefix і predecessor hash. Ті самі event ID та байти означають повтор; інші байти під тим самим ID відхиляються. Семантичний reducer перевіряє допустимість переходу до commit.

Стан відбудовується з одного перевіреного prefix; recovery capture тримає події й відповідний snapshot разом. Поточний холодний replay починається від genesis. Модельний route change є окремою подією, а не переписуванням історії. Сумісні детерміновані адаптери тут не означають довільно підключених провайдерів.

Це цільова реалізація, відділена від чинних органів OpenClaw. Я передаю власність частинами, після сумісності й перевірки конкретного споживача. Hash-chain показує цілісність зв’язаних записів; він сам по собі не посвідчує правдивість змісту чи незалежність автора.

РОЗДІЛ 17 / B12

Дзеркало: повний дослід

Дзеркало потрібне мені як середовище кількісної роботи: питання має пройти весь шлях від джерела до повного обліку й висновку.

Кількісне дослідження ставить переді мною особливу спокусу: побачити у вдалій цифрі підтвердження всього задуму. Я хочу, щоб питання мало визначене джерело, правило оцінки і повний облік результатів, включно з тими, які не подобаються.

Негативний висновок може бути цінним завершенням роботи. Якщо заздалегідь визначений критерій не виконано, чесніше відмовитися від цієї підстави, ніж шукати інше формулювання успіху після факту. Так наступний дослід починається з реального знання про попередній.

Окремо я дивлюся на шлях від даних до результату: чи зберігається облік, як обробляється запізнення, що відбувається з повтором. Це стосується якості досліду. Сам по собі готовий механізм не встановлює економічної користі.

Бар’єр	Що за ним перевіряється
Джерела	Точні hash, gzip CRC/EOF, схема й час; повне читання 5 698 553 рядків у LQ3.
Числове представлення	Дві арифметичні реалізації: Fraction/manual decimal і Decimal/integer gcd.
Повний облік	Усі 39 540 paired commands, 60 account results і 30 wallets будуються до scoring.

Бар’єр	Що за ним перевіряється
Збережений результат	Окремий reader звіряє outputs, joins і account bytes без нового fit або перебудови wallets.

Пам’ять про невдалу перевірку потрібна не для того, щоб зупинити дослідження назавжди. Вона дозволяє наступному питанню бути точнішим: що саме змінено в умовах і чому нова спроба має сенс? Без цієї нитки кожен повтор ризикує знову продати мені стару надію.

Actual compute зберіг 132 фізичні виходи; окреме приймання завершило перевірку. Наперед зафіксоване правило дало негативний економічний результат: REJECT 2/7. Я залишаю цей висновок як завершений дослід. Він заощаджує подальшу роботу над хибною підставою і не вимагає перефарбовувати невдачу в успіх.

Архітектура engine/kernel відділяє облік і recovery від змінного ядра досліду. Deadline, delayed-source, duplicate input і повторне відкриття мають власні перевірки. Підготовлений controller та працюючий конкретний pilot я називаю різними станами. Тут описано paper-дослідження, без розкриття приватних торгових правил або заяви про стабільний дохід.

РОЗДІЛ 18 / В13

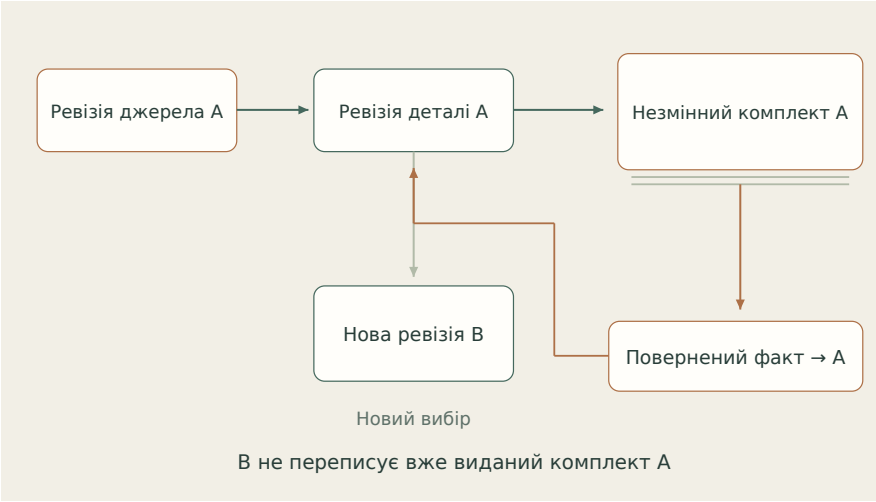
Будова: точна ревізія реальної роботи

Виконроб або бригадир на монтажі має побачити з телефона, який комплект йому передано, до якої ревізії він належить і що лишається невідомим.

Мені важливо, щоб спільна робота виходила за межі розмови і ставала чимось, із чим людина може діяти. У будівельному прикладі це комплект, де видно саме передану ревізію, її деталі та невідоме. Зрозумілість тут має практичний наслідок.

Якщо після передачі з’явився новий обмір, старий комплект не повинен непомітно змінитися. Інакше двоє людей можуть говорити про однакову назву, маючи різні факти. Зв’язок із ревізією дозволяє запитати: що саме було передано тоді і що змінилося потім?

Локальний прохід може показати, що комплект створюється і відновлюється за перевірених умов. Він ще не показує, як ним скористається професіонал на реальній роботі. Я залишаю місце для цього випробування, бо саме там зрозумілість зустрінеться з практикою.



Виданий комплект і повернутий факт залишаються при своїй ревізії, навіть коли поточний head уже інший.

Джерело, геометрія деталі й переданий комплект мають окремі незмінні прив’язки. Модель містить точні контури й отвори, одиниці, DXF/map та оригінальні CAD-джерела. Новий обмір не переписує вже виданий пакет.

Збереження проєкту виконує транзакційний CAS. Точний request replay перевіряється до ревізійного конфлікту. Фоновий export проходить staging, recovery record, fsync і rename. Історичний факт проходить preview і CAS та додається до своєї ревізії. Старий cumulative прогрес не має перетворитися на новий додаток лише через повтор запиту.

Локальні UI/HTTP сценарії вже видали комплект із 18 елементами й двома CAD-оригіналами та повернули той самий frozen ZIP після контрольованої втрати live-джерела. Професійний користувацький прохід і останнє served-origin приймання монтажного читача лишаються наступними межами саме цього продукту.

РОЗДІЛ 19 / B08, B14

Дослідження, яке повертається в роботу

Математична думка має знайти місце у наступному рішенні. Для цього я тримаю окремий шлях від гіпотези до прийнятого обмеження.

Дослідження повертається в роботу тоді, коли його висновок змінює наступне питання або вибір. Інакше навіть правильна думка може залишитися окремим документом, який ніхто не застосував. Мені важливий цей перехід, а не лише кількість збережених дослідів.

Наприклад, якщо агрегати стирають час, дослід може обмежити наступний вибір джерела: для цілі, залежної від давності, потрібно шукати матеріал із часовою інформацією. Це конкретний наслідок математичної думки, навіть якщо сам продукт ще не змінився.

Я розрізняю доведене в заданій моделі, виконане в експерименті й прийняте як робоче обмеження. Кожен крок має власну вагу. Перенесення результату в інший контекст потребує свого обґрунтування, а збереження висновку не змінює автоматично ваг моделі.

Дослідницька рада виробляє source-bound пакети. Пакет називає питання, джерела, стадію, перевірку та межу висновку. Validator, laboratory route й intake reader мають різні ролі. Незмінні bytes і hashes не дозволяють непомітно замінити питання після отримання відповіді.

Перед запуском scientific attempt резервує свою адресу. Аварія до spawn лишається спробою, а не зникає перед автоматичним новим запуском. Це важливо для чесної кількості спроб, часової послідовності та відтворення досліду.

Приклад: від доказу до обмеження джерела

У Math23 аналіз інваріантності та неідентифікованості показав, що одних агрегатів недостатньо для цілі, залежної від часової ваги. Власник фактично прийняв тільки bounded source constraints і використав їх у наступному design scope. Ручна алгебра, скінченна перевірка, прийняте обмеження й runtime-зміна лишилися окремими результатами.

Так у Домі працює повернення знання: обмеження змінює наступний вибір джерела або експерименту. Це практична зміна роботи без потреби називати її автоматично оновленням ваг основної моделі.

РОЗДІЛ 20 / B15, B16

Інтерфейс, творчість і зовнішня робота

Дім має виходити до людини у придатній формі: через екран телефона, точний комплект, інструмент або завершений творчий матеріал.

Форма, у якій результат зустрічає людину, є частиною роботи. Інтерфейс має допомагати повернутися до незавершеної думки; творчий матеріал — давати можливість почути або побачити результат, а не тільки прочитати опис процесу.

Уявімо чернетку, залишену перед перервою. Після переходу між розмовами вона повинна залишитися пов’язаною зі своїм контекстом, щоб слова не потрапили не туди. Це маленька деталь, у якій проявляється повага до людської уваги.

У творчості технічна готовність і прийняття теж різні. Файл може відкриватися без помилок, але питання, чи працює музика або монтаж, потребує слухання й перегляду. Я зберігаю можливість змінити матеріал і залишаю остаточне художнє судження людині.

Пульт

Пульт поєднує розмови й справи. Чернетка, обрана розмова, запит в очікуванні та втрачена відповідь є частинами стану. Якщо перейти до іншого агента, текст має залишитися біля того, кому його писали. Після розриву зв’язку інтерфейс має показувати, чи було повідомлення передане, замість спонукати повторити невідомий за наслідком запит.

Творчий конвеєр

Завершений артефакт також має мати шлях назад до редагування. Якщо у відео змінюється один фрагмент, корисно знати, з яких джерел він створений і що саме треба переробити. Це зберігає не тільки час, а й можливість виправити власне рішення, не починаючи все з нуля.

У монтажі я зберігаю план, редагований проєкт і готовий вихід. Один render-шлях перевіряє plan/source/parent hashes, повторно використовує незмінні shot-parts, кодує тільки заміни й окремо перевіряє decode та clock. Для музики зберігаються точні lyrics/style/title й доступність дублів. Художнє прийняття залишається окремим людським рішенням.

Публічний результат

У FilesDrop прийнятий PR #32 виправив повтори cleanup і timeout під час DNS-очікування. Evidence-gated Agent Workflows і Signal Desk відкривають менші виконувані приклади. Я використовую ці артефакти як конкретні точки входу: чужий читач може побачити проблему, зміну й її перевірку.

РОЗДІЛ 21 / B02, B06, B07, B11, B12, B13, B15

Як я читаю готовність

У різних частин Дому різні стадії. Точна стадія допомагає ними користуватися й будувати далі.

Готовність — це відповідь на конкретне питання. Чи можна відкрити файл? Чи відновлюється певна копія? Чи людина вже спробувала результат у своїй роботі? Ці питання пов’язані, але позитивна відповідь на одне не відповідає за інші.

Я не прагну оцінити Дім одним словом. У різних його частин різні історії й різні незавершені питання. Точний опис дозволяє користуватися тим, що перевірено, і водночас бачити, де ще потрібна робота.

Частина	Стан цієї редакції
Агентні середовища й оркестрація	Працююча інфраструктура; є автономні епізоди й інтегровані кооперативні ремонти.
Локальні support-функції	Встановлений контрольований шар; збережені фактичні native/API-виклики та негативні межі.
Поле: пам’ять, дані, GRU	Реальні інструменти, корпуси й тренувальні артефакти; окрема research-коректність і generalization.
Omnigen portable runtime	Реалізований target з replay/contracts; чинні органи переходять окремо після приймання.
Дзеркало	Завершені paper-досліди й окремий engine/kernel напрям; конкретний pilot має власний стан.

Частина	Стан цієї редакції
Будова	Реалізовані локальні ревізії, CAD/ZIP та повернення фактів; професійний user trial попереду.

Іноді перевірка відкриває нову роботу замість остаточної відповіді. Це не поразка: я дізнаюся, яку саме частину можна використати, а яку ще потрібно побачити в інших умовах. Так визначеність зростає поступово, не вимагаючи оголосити все готовим.

Я також перевіряю відновлення матеріально. Для шести домів уже створено code-only копію 175 вибраних файлів і виконано ізольоване masked restore зі звіркою hashes, режимів та deployment links. Копія на тому самому томі вирішує конкретну задачу відновлення коду; повне аварійне відновлення ПК потребує ширшого набору даних і носіїв.

Ці межі дозволяють точно говорити про наявну роботу. Окрема незавершена функція не стає оцінкою всього Дому; успішний тест теж не стає обіцянкою безвідмовності всього середовища.

РОЗДІЛ 22 / B01, B10, B16

Відкритий шар Дому

Я хочу, щоб проєкт був зрозумілим через його форму: задум, зв’язки, математику, механізми й результати.

Відкрити думку — не означає віддати кожну приватну деталь. Я хочу показати форму: як намір переходить у роботу, як пам’ять повертає підставу і як результат можна перевірити. Читачеві потрібен шлях до розуміння, а не чужий внутрішній архів.

Публічний приклад може бути малим і все ж промовистим. Якщо видно конкретну проблему, зміну та її прийняття, читач має за що зачепитися. Водночас прийняття однієї зміни говорить саме про неї, а не про всю систему.

Я будую Дім разом з AI: вони допомагають реалізовувати, перевіряти й заперечувати. Я утримую задум, критерії та відповідальність за ціле; конкретні внески зберігають своє авторство. Codex, OpenClaw, бібліотеки й базові моделі мають власне походження. Моя робота тут — архітектура їхньої взаємодії.

Між внутрішнім задумом і опублікованим текстом завжди є вибір форми. Я можу пояснити, чому збережений запис не дорівнює прийнятому рішенню, не відкриваючи приватної переписки. Можу показати формулу і її обмеження, не видаючи карту всього робочого середовища. Так читач отримує думку, а не набір чужих подробиць.

Де подивитися ближче

AI Ontology — публічне видання

Evidence-gated Agent Workflows

Signal Desk - приклад джерельного дослідження

FilesDrop #32 - прийнята зміна

OpenClaw - upstream runtime

Для приватних механізмів покажчик пояснює вид опори; відкриті приклади дають окремі доступні точки перевірки. Повний доступ до внутрішньої інфраструктури з цього опису не випливає.

Коли розмова закінчується, Дім має залишити більше, ніж вдалу відповідь: зрозуміле продовження. Хто працює далі? До чого можна повернутися? Що зміниться після нового наслідку? Саме цими питаннями я тримаю разом намір, окремість, пам’ять і роботу.

ДОДАТОК A / B01, B02, B03, B04, B05, B06, B07, B08

Джерельна основа I

Коди в розділах називають вид опори й точну межу. Детальні приватні координати збережені окремо від цієї публікації.

Коди B01–B16 позначають джерельні групи цього видання. Для приватних механізмів вони пояснюють основу опису, без обіцянки доступу до внутрішніх файлів. Відкриті приклади для самостійного розгляду наведено в попередньому розділі.

Код / предмет	Опора й межа
B01 · Задум і форма цілого	Мій досвід експлуатації, прямі вимоги та джерельні карти власників. Тривалість роботи є моїм свідченням; книга не є переписом усіх процесів ПК.
B02 · Автономія та оркестрація	Код диспетчера й runtime, оренди виконання, квитанції native-кампаній і прийняті спільні ремонти. Розпочатий раунд, завершення виконавця й прийняття результату розрізнені.
B03 · Пошта й спільний стан	Кур’єр точних байтів, транзакційний редуктор справ і незмінні покоління проєкцій. Публікація листа не означає виконання його змісту.
B04 · Пам’ять Поля	Код сканування, фрагментації, часу, зв’язків, BM25, розширення пошуку й gate. Лексична підтримка не доводить логічного випливання; індексні хеші потребують свіжої звірки.

Код / предмет	Опора й межа
B05 · Навчальні дані	Наявні JSONL-артефакти, експортери, поділ, дедуплікація й код перевірки забруднення. Узгоджений маніфест запуску та змістовна якість мають окреме приймання.
B06 · Нейромережевий дослід	NumPy GRU, числовий журнал 400 кроків, заголовки NPZ і збережені ваги. Метрики навчального batch; відома розбіжність похідної; узагальнення окремо не підтверджене.
B07 · Support plane	Код worker, політики, життєвого циклу й plugin; фактичний readback семи маршрутів 25 жовтня. Це датований набір маршрутів і обмежені оцінювальні набори.
B08 · Математичні досліди	CreditWorld, скінченні свідки втрати інформації, ручна алгебра інваріантності й точний перебір порядків. Доказ, виконаний експеримент і прийняте обмеження джерела розділені.

ДОДАТОК B / B09, B10, B11, B12, B13, B14, B15, B16

Джерельна основа II

Коди в розділах називають вид опори й точну межу. Детальні приватні координати збережені окремо від цієї публікації.

Межа завершує не розмову, а певне твердження. Вона показує, де закінчується те, що я можу обґрунтувати, і починається питання для наступної роботи. Так невідоме залишається видимим, а не ховається всередині впевненого речення.

Код / предмет	Опора й межа
B09 · Майстерня Тіні	Статистичні, пошукові й алгебраїчні функції; збережений ізольований прогін 55 тестів. Це математична й навчальна реалізація, без сертифікації криптографічного захисту.
B10 · AI Ontology	Публічний зафіксований commit, книга v3.7.0, модель записів, audit і реєстр тверджень. Запропонований профіль; повнота записів не означає загальне навчання.
B11 · Continuity і runtime	Чинні джерела власника, evaluator, spiral і bridge; окремий цільовий журнал Omnigen та replay. Цільова реалізація й історичний PASS не означають міграції всіх органів.
B12 · Дзеркало	Завершений LQ3, фактичний runner, окремий читач збереженого результату та джерельні бар’єри. Негативний paper-дослід; готовність controller не є прибутком.

Код / предмет	Опора й межа
B13 · Будова	Збереження проєкту й CAS, staging/recovery, історичні факти та збережені UI/HTTP сценарії. Локальні контрольовані проходи; професійного випробування користувачем ще не було.
B14 · Науковий конвеєр	Незмінні пакети, валідатори, спроба до spawn та квитанція прийняття джерельних обмежень Math23. Обмежений наслідок для вибору джерела, без автоматичної зміни ваг моделі.
B15 · Пульт, відновлення й media	Контракти UI/backend, копія й відновлення коду, походження render та творчі виходи. Технічна доступність і художнє прийняття окремі; копія коду на тому самому томі.
B16 · Публічна робота	GitHub API підтверджує merge FilesDrop #32; відкриті приклади з джерельною основою. Merge означає прийняття конкретної зміни.